GAMECRAFT : BUILDING A 3D PLATFORMER GAME USING UNITY HUB

A VALUE ADDED COURSE PROJECT REPORT

Submitted by

SARVARSHNI S (211423244194)

SAMYUKTHA R (211423244181)

ROSHINI S (211423244173)

SUPRIYA V (211423244228)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BUSINESS SYSTEMS



PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

APRIL 2026

PANIMALAR ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this Value Added Course project report **"GAMECRAFT: BUILDING A 3D PLATFORMERS GAME USING UNITY HUB"** is the Bonafide work of **"SARVARSHNI S (211423244194), SAMYUKTHA R (211423244181), ROSHINI S (211423244173), SUPRIYA V (211423244228)"** who carried out the project work under my supervision.

SIGNATURE

Dr. C. BHARANIDHARAN, M.E., Ph.D.,
ASSOCIATE PROFESSOR
Mrs. S. MAHALAKSHMI, M.E., (PhD).,
ASSISTANT PROFESSOR
SUPERVISOR
DEPARTMENT OF AI & DS,
Panimalar Engineering College,
Chennai - 600123.

SIGNATURE

Dr. S. MALATHI, M.E., Ph.D.,
PROFESSOR,
HEAD OF THE DEPARTMENT,
DEPARTMENT OF AI & DS,
Panimalar Engineering College,
Chennai - 600123.

Certified that the above-mentioned students was examined in end semester VAC Project held on …….

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our respected Secretary and Correspondent **Dr. P. CHINNADURAI, M.A., Ph.D.,** for his kind words and enthusiastic motivation, which inspired us a lot in completing this project.

We express our sincere thanks to our directors **Tmt. C. VIJAYARAJESWARI, Dr. C. SAKTHI KUMAR, M.E., Ph.D., and Dr. SARANYA SREE SAKTHI KUMAR B.E., M.B.A., Ph.D.,** for providing us with the necessary facilities to undertake this project.

We also express our gratitude to our Principal **Dr. K. MANI, M.E., Ph.D.,** who have facilitated us in completing the project successfully.

We thank the Head of the AI&DS Department, **Dr. S. MALATHI, M.E., Ph.D.,** for the support extended throughout the VAC project.

We would like to thank our supervisor, **Dr. C. BHARANIDHARAN, M.E., Ph.D., Mrs. S. MAHALAKSHMI, M.E., (PhD)** and all the faculty members of the Department of Artificial Intelligence & Data Science for their advice and encouragement for the successful completion of the VAC project.

We also wish to express our heartfelt thanks to our parents and friends for their unwavering support, patience, and constant encouragement throughout the duration of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	1
1	INTRODUCTION	3
1.1	Scope of the Project	4
1.2	Problem Statement	6
1.3	Objectives	8
2	SOFTWARE REQUIREMENTS	12
2.1	Programming Languages	12
2.2	Tools and Technologies	13
3	PROJECT DETAILS	16
3.1	AR Module Overview	16
3.2	VR Module Overview	18
4	PROJECT DEVELOPMENT	20
4.1	Unity Hub Setup	20
4.2	Scene Design	22
4.3	Prefab and Asset Configuration	23
5	IMPLEMENTATION	26
5.1	AR Implementation	26
5.2	VR Implementation	28
5.3	C# Scripting	30
6	DEBUGGING	33
6.1	Common Errors and Fixes	33
6.2	Testing on Device	35
7	TESTING	38
7.1	Unit Testing	38
7.2	Integration Testing	41
7.3	User Acceptance Testing	42
8	OUTPUT	44
9	CONCLUSION AND FUTURE WORK	47

9.1	Conclusion	47
9.2	Future Work	48
	REFERENCES	50

ABSTRACT

GameCraft: A Comprehensive 3D Platformer Development Project

GameCraft is an intricately designed, fully functional two-level 3D platformer game created from the ground up using Unity Hub and the Unity Editor 2022.3 LTS. This project is a hallmark of a Value Added Course (VAC) on game development technology offered at Panimalar Engineering College. It showcases the entire lifecycle of building an interactive 3D game, from the initial setup of the development environment to the delivery of the final executable, utilizing only freely available tools under the Unity Personal License, complemented by Visual Studio Community 2022.

At the heart of GameCraft is a player-controlled capsule character, which embarks on a thrilling journey through two increasingly challenging levels. These levels are meticulously designed with a combination of static and dynamic platforms, along with patrolling enemy cubes and collectible coin spheres. The game relies solely on Unity's built-in primitive shapes—specifically Plane, Cube, Capsule, and Sphere—to construct all 3D geometry. This focused approach hones in on programming logic, the architecture of game systems, and the robust Unity component model, while consciously avoiding the complexities of external asset creation.

The movement of the player character is implemented through direct manipulation of the Rigidbody's velocity, responding intuitively to keyboard inputs (WASD and arrow keys). To ensure a smooth gaming experience, a Physics.CheckSphere grounding system is integrated, preventing the possibility of double-jumping and enhancing the player's control. Furthermore, a follow camera system is cleverly implemented in the LateUpdate method, utilizing Vector3.Lerp interpolation to maintain a seamless tracking experience of the player.

The game's enemy AI is designed using the Vector3.MoveTowards method for waypoint patrols, creating a more engaging and challenging environment. Level 2 introduces moving platforms that leverage Vector3.Lerp with player Transform parenting, adding another layer of complexity and excitement to the gameplay.

The audio components of GameCraft are managed by a persistent AudioManager singleton utilizing the DontDestroyOnLoad function. This allows for the delivery of immersive looping background music, alongside PlayOneShot event-driven sound effects

that enhance the gaming experience—covering essential events such as jumping, coin collection, enemy defeat, game over, and level completion.

The game's user interface (UI) is meticulously designed to include a Start Screen, a live HUD that displays the score and coin count via TextMeshPro, a Level Complete screen that presents the cumulative score, and a Game Over screen equipped with options for retrying or returning to the main menu. All these elements are interconnected through the UIManager's callback functions, effectively creating an intuitive and engaging user experience.

The entire project successfully meets an extensive array of objectives regarding character-level recognition. Key features include physics-based movement, comprehensive grounding checks, an effective follow camera system, moving platforms, enemy AI with stomp and kill detection, collectible scoring mechanisms, audio management, and a fully integrated multi-scene UI system.

GameCraft was developed by following a structured ten-part reference series on 3D game development in Unity. This series provided a clear, step-by-step pedagogical framework for introducing each system, which significantly aided in the development process. The project serves as a testament to the idea that a polished and complete 3D game can indeed be achieved within an academic setting using entirely free tools.

Through GameCraft, the project not only highlights the potential of independent game development using Unity but also exemplifies the practical application of theoretical concepts learned in the classroom. It stands as a demonstration of both creativity and technical skill, proving that with determination and the right tools, aspiring game developers can create compelling interactive experiences.

Keywords: Unity Hub, Unity 2022.3 LTS, 3D Platformer, C# Scripting, Rigidbody Physics, Enemy AI, Waypoint Patrol, AudioManager, SceneManager, TextMeshPro, Game Development, Value Added Course.

CHAPTER 1
INTRODUCTION

Video game development represents a unique convergence of multiple disciplines, including software engineering, applied mathematics, real-time systems design, and interactive media. Creating a functional 3D game from scratch requires developers to deftly manage a myriad of complex tasks simultaneously. This includes physics simulation, three-dimensional coordinate transformations, user input processing, artificial intelligence, audio engineering, and graphical user interface design. All these elements must operate seamlessly within a continuous real-time loop, ideally executing at sixty frames per second or higher. For students pursuing degrees in computer science and engineering, the multifaceted nature of game development offers one of the most technically comprehensive and educationally enriching project experiences available in an academic setting.

Unity stands out as the world's most widely adopted real-time 3D development platform. It is the driving force behind more than fifty percent of all mobile games released globally, and it supports a substantial percentage of titles across various platforms, including PC, console, and WebGL. Unity Technologies designed the engine around a component-based entity architecture, in which game functionality is built by attaching focused, reusable C# script components to generic GameObject containers. This design philosophy promotes modularity, separation of concerns, and code reuse — principles that extend well beyond game development and are directly applicable to enterprise software engineering and complex systems design.

The Unity Hub serves as the centralized desktop management application, acting as the gateway for all Unity development activities. This unified interface simplifies the installation and management of multiple versions of the Unity Editor. It also facilitates project creation and organization, license activation, and the downloading of platform-specific build support modules. For students and novice developers, Unity Hub effectively removes the configuration complexity that often accompanies Unity setup, allowing them to establish a complete development environment within just under an hour of initial installation.

Visual Studio Community 2022, the free Integrated Development Environment (IDE) provided by Microsoft, integrates seamlessly with the Unity Editor to enhance the development experience. It offers robust features such as C# IntelliSense, syntax

highlighting, real-time error detection, and a powerful step-through debugger that can attach directly to the running Unity process. This integration positions Unity and Visual Studio as the industry-standard toolchain for game development, catering to everyone from student projects to large-scale AAA commercial productions.

The GameCraft project emerged as a Value Added Course (VAC) initiative at Panimalar Engineering College, aimed at introducing students enrolled in the Computer Science and Business Systems program to the complete lifecycle of game development. The project is structured around a comprehensive ten-part reference series focused on building 3D games in Unity, with each segment introducing a new game system to the evolving project. The outcome is a two-level 3D platformer that features physics-based player movement, a smooth follow camera, patrolling enemy AI, collectible coins, moving platforms, a sophisticated audio system, and a well-designed multi-scene user interface.

This technical report meticulously documents every phase of the GameCraft project's development, providing a thorough account of the processes involved. It covers the installation and configuration of the development environment, offers a step-by-step guide to constructing both game levels, and details the implementation of all twelve C# scripts, complete with in-depth technical explanations. The report also highlights the systematic debugging of significant issues encountered throughout development and outlines comprehensive unit and integration testing procedures. Furthermore, all game outputs are fully documented. Organized into nine chapters corresponding to the major phases of the software development lifecycle, this report is intended as a fully replicable reference for future students embarking on their own game development projects within the department, facilitating their journey into the exciting world of game creation.

1.1 Scope of the Project

The scope of the GameCraft project encompasses a comprehensive approach to the design, development, testing, and documentation of a two-level 3D platformer game utilizing Unity Hub 3.x and Unity Editor 2022.3 LTS. This project is intentionally developed within the constraints of freely available software and standard consumer hardware, demonstrating that professional-quality interactive 3D software development is not only viable but also accessible to students without incurring any commercial

software expenditure. This initiative serves as a practical demonstration of how high-quality game development can be achieved in educational contexts.

From a gameplay perspective, the project offers an engaging and fully playable two-level game that features a well-structured and polished game loop. Players start at a Start Screen before progressing through Level 1, where they collect six coins while navigating static platforms and avoiding or stomping two patrolling enemies. Upon completing Level 1, players transition to Level 2, which presents a more challenging environment, replete with ten coins, four enemies, and two moving platforms. The game concludes with players either reaching a Level Complete screen, which indicates that they have successfully collected all the coins in the level, or a Game Over screen triggered by contact with an enemy from the side or front. To enhance the experience, the score accumulated across levels is maintained by a persistent GameManager singleton, providing a cohesive scoring system.

From a technical perspective, the project scope thoroughly explores various aspects of game mechanics and programming. It covers physics-based character movement utilizing Unity's Rigidbody component, implemented with direct velocity control in the FixedUpdate method. The mechanics enforce single-jump functionality using Physics.CheckSphere for grounding detection, while a smoothly following camera employs Vector3.Lerp in the LateUpdate method to create an immersive experience. Additionally, the game features waypoint patrol AI, which is executed using Vector3.MoveTowards with a configurable patrol speed, and includes moving platform functionality driven by Vector3.Lerp alongside player Transform parenting. Trigger-based coin collection is implemented to allow for effective score management, and enemy interactions are differentiated by zones—using a top trigger for stomp actions and a body collider for kill mechanisms. Sound management is handled through a persistent AudioManager singleton that supports concurrent sound playback via PlayOneShot, and players can navigate multi-scene environments effortlessly through the SceneManager.

All 3D geometry utilized within the project is constructed strictly from Unity's built-in primitive shapes, such as Plane objects for ground surfaces, Cube objects for platforms and enemies, and distinct models like Capsule for the player and Sphere for collectible coins. The decision to refrain from using external 3D modeling software, Asset Store purchases, or third-party frameworks was deliberate, ensuring that the focus remains on

fundamental programming logic, game system architecture, and the effective use of the Unity component model, rather than on asset creation pipelines or visual production skills, which could divert attention from the core educational objectives of the project.

In terms of project boundaries, the scope explicitly excludes several features that are traditionally associated with more complex game development. These exclusions include multiplayer networking, 3D character models or skeletal animations, procedurally generated levels, mobile platform builds, online leaderboards, save-game systems, shader programming, and post-processing visual effects. It is vital to note that these exclusions are not indicative of limitations imposed by the platform or tools; rather, they represent deliberate scope boundaries that align well with the objectives of a Value Added Course project. All identified excluded features are recognized as potential future work items, with implementation strategies thoroughly outlined in Chapter 9 for interested developers or students wishing to expand upon this foundational project.

1.2 Problem Statement

In the richly designed universe of GameCraft, audio is not merely a feature—it is the lifeblood that courses through every aspect of the gameplay experience. The orchestrated management of this intricate soundscape is masterfully handled by a persistent AudioManager singleton, which implements the robust DontDestroyOnLoad function. This strategic choice establishes a foundation where the AudioManager can maintain its presence throughout the game, seamlessly transitioning players from one scene to another without interruption, thereby ensuring that the auditory experience remains consistent and immersive.

At the heart of this auditory design is a carefully composed backdrop of looping background music that is crafted to resonate with the various themes and emotions underpinning the game. Each piece of music is not just a soundtrack; it is a nuanced narrative device that draws players deeper into the fabric of the game world. From exhilarating, upbeat tracks that encourage exploration and adventure to haunting, atmospheric melodies that convey suspense and mystery, the background music is a dynamic entity that evolves with the player's journey. This musical accompaniment is

meticulously designed to synchronize with the gameplay, enhancing moments of triumph and reflection and enriching the player's emotional engagement.

In harmony with the enriching background scores, the AudioManager features a comprehensive suite of event-driven sound effects that perplex and delight through the implementation of the PlayOneShot method. This technique ensures that each notable action within the game is punctuated by a unique auditory cue, creating a lively and responsive environment. Players can hear the satisfying thud of their character jumping, a crucial action that not only amplifies their sense of agency but also instills a feeling of fluidity and control within the game mechanics.

The act of collecting coins transforms into an exhilarating experience, as the delightful chime resonates with each successful pickup—rewarding players for their agility and strategic maneuvering in a gratifying manner. When vanquishing enemies, dynamic sound effects provide an auditory celebration of victory, reinforcing the achievement and providing players with a sense of impact and accomplishment. Furthermore, during moments of failure, the poignant soundmark accompanying a game over evokes a breath of reflection, serving as a gentle reminder of the challenges to overcome and the journey yet to unfold.

Every completed level is celebrated with an uplifting fanfare, an auditory marker that signifies not just progress but also the player's dedication and skill. This triumphant sound ignites a sense of elation, encouraging players to push further and explore the depths of GameCraft's multifaceted realms.

The blending of immersive background music with vibrant sound effects creates a deeply engaging tapestry that envelops players in the story. The AudioManager's thoughtful orchestration ensures that the audio landscape is always in sync with the on-screen action, effectively heightening the drama and stakes of each moment. The result is an ever-responsive audio experience that captivates the senses—drawing players into a realm where sound and gameplay are intricately intertwined.

GameCraft's commitment to its auditory experience showcases a recognized understanding that sound is not just an accessory; it is a fundamental component of storytelling and player engagement. Through this sophisticated design approach, the game cultivates an atmosphere in which every audio cue resonates with purpose, creating layers of meaning and involvement. Each player action reverberates through this soundscape, transforming

the game into a living, breathing entity that reflects the triumphs, challenges, and journeys of its players.

In conclusion, the audio framework within GameCraft stands as a testament to the power of sound in gameplay, weaving together music and effects in a symbiotic relationship that elevates the entire experience. It creates a rich, immersive environment where every sound encapsulates the very essence of adventure, ensuring that players are not just participants in the game, but are woven into a narrative that captures their hearts and minds, leaving a lasting impression as they embark on their unforgettable gaming journey.

1.3 Objectives

In GameCraft, the audio experience is expertly controlled by a robust AudioManager singleton that utilizes the DontDestroyOnLoad function, ensuring that audio elements remain consistent across scenes and creating a seamless auditory journey for players. This system delivers immersive looping background music that enriches the gaming atmosphere while also employing dynamic PlayOneShot sound effects that activate during key events, amplifying player engagement. These sound effects enhance essential moments such as the exhilarating sound of jumping, the satisfying chime of coin collection, the victorious clang of enemy defeat, the somber notes that signify game over, and the triumphant music that plays upon level completion. By thoughtfully integrating these audio components, GameCraft not only elevates gameplay but also transforms routine actions into memorable experiences, making every moment resonate with significance.

Objective 1 — To create an effective development environment, it's essential to install and properly configure Unity Hub version 3.x, the Unity Editor 2022.3 LTS, and Visual Studio Community 2022. This setup should include enabling full C# IntelliSense support on your Windows 10/11 system, as this feature significantly enhances coding efficiency. Taking the time to establish each component correctly is key to facilitating a smooth workflow. Additionally, before diving into any development work, it is crucial to verify the entire toolchain to ensure all components are functioning harmoniously. This thorough preparation will ultimately streamline your development process and help prevent potential issues as you progress with your project.

Objective 2 — To successfully initiate a Unity project utilizing the 3D Built-In Render Pipeline template, it is essential to carefully set up your project by organizing four critical scenes: StartScreen, Level1, Level2, and GameOver. Each scene needs to be assigned the appropriate build index within the Build Settings to ensure smooth transitions throughout the gameplay experience. Furthermore, it is vital to establish a logical and intuitive project folder structure, which will help maintain organization and efficiency during development. Create main folders for Scripts, Scenes, Materials, Audio, and Prefabs to streamline your workflow and make asset management easier as your project expands. With these foundational elements thoughtfully arranged, you will be well-equipped to embark on the detailed construction of your game scenes.

Objective 3 — To effectively implement the PlayerMovement.cs script in Unity, the focus should be on utilizing the Rigidbody component for optimal player movement. This involves translating player input from the WASD keys and arrow keys into smooth, framerate-independent 3D movement. By directly assigning velocities in the FixedUpdate method, we ensure consistent movement regardless of frame rates. Additionally, incorporating a grounding check for single jumps can be achieved through the Physics.CheckSphere method, which allows us to accurately determine if the player is on the ground. For executing the jump, we utilize Rigidbody.AddForce with ForceMode.Impulse, enabling a responsive and dynamic jumping experience. This approach provides a solid foundation for creating fluid player controls in a 3D environment.

Objective 4 — The implementation of the CameraFollow.cs script is essential for creating a smooth and immersive camera experience in gameplay. It begins by calculating a persistent offset vector in the Start method, which establishes a consistent distance and angle between the camera and the player. As the game progresses, the camera's position is updated each frame in the LateUpdate method, utilizing Vector3.Lerp interpolation to ensure a fluid transition that minimizes any camera jitter. This approach is particularly important as it maintains the correct execution order relative to player movement, allowing for a seamless visual experience that enhances the player's immersion in the game world. By effectively managing the camera's positioning and movement, we can create a more engaging and enjoyable gameplay experience.

Objective 5 — To create a dynamic and smooth movement system for platforms, we will implement the MovingPlatform.cs script using Vector3.Lerp, which allows for precise interpolation between two points in space based on a journeyFraction variable. This

approach ensures that platforms transition smoothly back and forth, enhancing player immersion. Additionally, we will incorporate player Transform parenting upon collision contact, effectively tethering the player to the moving surface and preventing any sliding or disorientation as the platform shifts. This method not only elevates the gameplay experience but also maintains a consistent sense of motion for players navigating these dynamic environments.

Objective 6 — In order to develop the EnemyPatrol.cs script, we will utilize Vector3.MoveTowards to facilitate a smooth and consistent-speed movement for the enemies as they patrol between predefined waypoints. Additionally, we will implement a dual-zone enemy interaction system to enhance gameplay dynamics. This will include employing a top trigger collider through the EnemyTop.cs script to detect stomp interactions and a body collider via the PlayerHealth.cs script to manage player-kill events effectively. By creating these systems, we can ensure a more engaging and responsive experience for players as they navigate through enemy territories.

Objective 7 — To enhance the gameplay experience, we will implement collectible coin objects featuring a rotation animation handled by CoinRotate.cs. This visual effect will attract players, making the coins appear dynamic and engaging. The collection mechanics will be managed through CoinPickup.cs, which will utilize trigger-based detection to register when a player collects a coin. All of this functionality will be seamlessly integrated with a singleton instance of GameManager.cs. This central manager will be responsible for maintaining essential game states, including the player's score, total coin count, and level completion status, ensuring that this information persists across scene loads and provides a smooth transition as the player progresses through the game.

Objective 8 — To create a robust audio experience in the game, we will implement a persistent singleton class called AudioManager.cs, utilizing the DontDestroyOnLoad feature to ensure it remains active throughout various scenes. This AudioManager will be responsible for delivering a seamless background music experience through a dedicated AudioSource that loops continuously, enhancing the overall atmosphere of the game. In addition, we will incorporate a separate AudioSource for sound effects, utilizing the PlayOneShot method to trigger event-driven sounds that coincide with major game events. By taking this approach, we can guarantee that both the background music and sound effects work harmoniously together, enriching player immersion and maintaining a dynamic audio environment throughout gameplay.

Objective 9 — User Interface: In order to create an engaging user experience, we will design and implement a Start Screen featuring a prominent Play button that invites players to jump into the game. Additionally, we will develop an in-game HUD that showcases a live TextMeshPro display for both the score and coin count, ensuring players are consistently informed of their progress. Upon completing a level, players will encounter a Level Complete screen that presents their final score along with a clearly labeled Next Level button to encourage continued play. Furthermore, a Game Over screen will be included, offering options to Retry the level or return to the Main Menu, thus providing players with a seamless navigation experience. All of these elements will be managed through dedicated per-scene UIManager.cs scripts to ensure smooth and efficient transitions between different game states.

Objective 10 — Documentation: In order to create a detailed and comprehensive technical project report that thoroughly documents the entire development process, it is essential to include all C# scripts accompanied by clear technical explanations. Additionally, the report should provide a meticulous account of all bugs encountered throughout the project, complete with root cause analyses and corresponding fixes. To ensure the robustness of the project, testing results must be presented with pass/fail records, demonstrating the efficacy of the implemented solutions. Finally, the output documentation should be thorough and well-organized, serving as a reliable and fully replicable reference for future students, thereby enhancing their learning experience and facilitating smoother project execution.

CHAPTER 2

SOFTWARE REQUIREMENTS

This chapter offers an in-depth and comprehensive specification of the various software tools, frameworks, and system configurations indispensable for the successful development of the GameCraft project. With a firm commitment to accessibility, the project utilizes exclusively freely available, no-cost tools, ensuring that the entire development environment is within reach for any student equipped with a standard consumer laptop and a reliable internet connection. Each tool is meticulously documented, encompassing crucial information such as its version, functional role within the project, licensing status, and essential configuration details that are specifically tailored to meet the unique requirements of GameCraft. This thorough documentation not only facilitates a clearer understanding of the development process but also empowers aspiring developers to engage with and replicate the project effortlessly.

2.1 Programming Languages

C# 9.0 (C-Sharp) serves as the exclusive programming language for the GameCraft project, making it central to the development process. As the official scripting language supported by the Unity engine, C# operates through Unity's integrated Mono runtime, following the .NET Standard 2.1 profile. Microsoft designed C# to be a statically typed, strongly object-oriented language, boasting comprehensive support for key programming features such as generics, delegates, events, lambda expressions, LINQ, and asynchronous programming with async/await. These powerful capabilities are particularly beneficial in the context of game scripting, allowing developers to create complex and efficient game mechanics effortlessly.

Unity’s incorporation of C# provides a rich set of game-specific application programming interfaces (APIs) encapsulated within the UnityEngine namespace. Every game behavior is implemented through classes that derive from UnityEngine.MonoBehaviour, the foundational class for component scripts in Unity. MonoBehaviour offers a structured lifecycle of event-driven callback methods that Unity invokes automatically at specific stages of the game loop. This lifecycle includes essential methods such as Awake(), which is triggered once during the initialization of the GameObject before Start(); Start(), which

occurs just before the first frame update following Awake(); and Update(), called every frame rendered. Additionally, FixedUpdate() is executed at a consistent physics timestep, typically 50 times per second, while LateUpdate() runs after all Update() calls in each frame. Other vital methods like OnCollisionEnter() manage interactions when colliders contact one another, and OnTriggerEnter() handles situations where a trigger collider overlaps with another collider.

Throughout the project, various C# language features are utilized extensively to enhance development efficiency and maintainability. For instance, the [SerializeField] attribute allows private fields to be visible in the Unity Inspector without exposing them publicly, ensuring encapsulation. Generic methods, such as GetComponent<T>(), enable developers to retrieve components without the need for explicit type casting, streamlining the coding process. The null-conditional operator (?.) offers a safe way to dereference components, preventing potential null reference exceptions. String interpolation (e.g., $"Score: {score}") provides a clean and readable approach for formatting UI text, enhancing user experience. Additionally, the use of IEnumerator coroutines with yield return new WaitForSeconds(delay) effectively facilitates the implementation of time-delayed game events, such as managing the pause prior to transitioning to the Game Over scene after a player's death.

It is worth noting that no other programming languages are utilized within the project. Although Unity's visual scripting system was explored, it was ultimately not adopted due to the superior control, performance, and debugging support that C# scripting offers. For graphical effects, the project relies solely on Unity's built-in Standard Shader, which negates the need for custom HLSL shader code, simplifying the shader management process and allowing for smooth visual enhancements without additional complexity.

2.2 Tools and Technologies

Unity Hub 3.x serves as an essential project management desktop application, available for download from unity.com/download. It acts as the gateway for all activities related to Unity development. With Unity Hub, users can install and manage various versions of the Unity Editor, create and open projects, activate licenses, and download necessary platform build modules. The application is free to download and use, providing an accessible entry point for both new and experienced developers. When activated through

Unity Hub, the Personal License grants full access to all features of the Unity engine, ensuring that users can take advantage of the platform to its fullest potential, with limitations only tied to the annual revenue thresholds outlined in Unity's Terms of Service.

At the heart of Unity development is the Unity Editor 2022.3 LTS (Long Term Support), which is the game engine and primary environment for development. This LTS designation signifies that Unity Technologies is committed to releasing ongoing bug fixes and security patches for this version for a minimum of two years. This makes it the most stable and recommended option for projects prioritizing reliability and long-term maintenance. The Unity Editor 2022.3 boasts several critical features, including the Scene View, which provides an interactive 3D editor; the Game View for real-time game previews; and the Hierarchy window, which displays the scene object tree. Additionally, users can utilize the Inspector panel for component property editing, the Project window for asset browsing, and the Console window for runtime log output, thereby creating a comprehensive development experience that covers all aspects of game creation.

For coding and scripting, Visual Studio Community 2022 stands out as the integrated development environment (IDE) tailored for C#. It directly integrates with Unity through the Unity Visual Studio Tools package, significantly enhancing the coding experience. This integration offers complete IntelliSense code completion for all UnityEngine types and methods, ensuring developers can write code more efficiently. Moreover, it features real-time syntax error highlighting that helps catch mistakes early in the development process. An invaluable feature is the debugging session capability, which enables developers to attach to the running Unity Editor process for detailed step-through inspection of script execution. As a bonus, Visual Studio Community is available for free to students and individual developers, making it an excellent resource for those starting in game development.

Rendering in Unity is primarily handled by the Built-In Render Pipeline, selected for its simplicity and ease of use. Unlike other rendering options like the Universal Render Pipeline (URP) and the High Definition Render Pipeline (HDRP), the Built-In pipeline requires no complex configuration, allowing developers to get started quickly. It supports all essential visual features, such as real-time shadows, material properties, and directional lighting, right out of the box. The Standard Shader included in this pipeline offers adequate visual quality for creating 3D platformers developed using primitive shapes, making it a favorable choice for projects with limited configurations and

resources. Its low per-project configuration overhead ensures that developers can focus more on creativity and less on technical setup.

For physics simulation, Unity relies on its robust Physics engine, powered by NVIDIA PhysX 4.1, which takes care of all rigid body simulations within the game. Functions like Physics.CheckSphere are utilized for grounding detection, while Rigidbody.AddForce facilitates jump impulses. The OnCollisionEnter callbacks manage interactions between platforms and enemies, while OnTriggerEnter callbacks are essential for detecting coins and enemy stomps. The Physics API, backed by PhysX, guarantees consistent performance, supported by a fixed physics timestep of 0.02 seconds (equating to 50 physics updates per second). This configuration ensures deterministic and stable physics simulation, effectively decoupling it from the rendering frame rate and enhancing overall game performance.

For user interface text rendering, Unity employs the TextMeshPro package, which replaces the legacy Unity UI Text component. TextMeshPro utilizes Signed Distance Field (SDF) font rendering technology, producing sharper, anti-aliased text at any resolution without the blurring or pixellation seen with traditional methods. This approach not only improves the precision of text rendering but has also established itself as the industry standard for modern Unity UI text in contemporary projects. TextMeshPro is conveniently included in the Unity Package Manager, allowing for easy integration into projects with no additional costs, thereby enhancing the visual quality of user interfaces.

Throughout the development process, version control is crucial for maintaining code integrity, and Git is the tool of choice in this regard. The project repository is initialized upon creation, with commits systematically made at the completion of each development milestone. To manage repository size efficiently, Unity provides a standardized .gitignore template that excludes generated directories such as Library, Temp, Obj, and Build from version tracking. This organization keeps the repository manageable and focused on essential project files. The version history that Git offers promotes safe experimentation during the debugging phase, allowing developers to roll back changes if any code modifications inadvertently introduce critical regressions. This version control practice enhances the stability of the development workflow, ensuring a reliable and efficient progression toward project completion.

CHAPTER 3

PROJECT DETAILS

This chapter presents a comprehensive technical specification for the GameCraft project, serving as a crucial foundation for all subsequent implementation and development efforts. Within this section, we delve into the intricate details of the scene architecture, providing a clear framework that supports the overall game design. We will explore the design of GameObjects, outlining their characteristics, behaviors, and how they interact with one another within the game world. An extensive inventory of scripts will be documented, detailing their functions, triggers, and the roles they play in shaping gameplay experiences. Moreover, this chapter will cover the nuanced configurations of various game components, emphasizing how they can be customized and optimized for performance. Readers will gain insight into the interactions between different game systems, illustrating the interconnectedness of elements that contribute to the game's dynamic environment. Overall, this comprehensive overview not only sets the stage for the technical aspects of GameCraft but also ensures that all subsequent sections can build upon this solid architectural foundation, leading to a more cohesive and well-structured development process.

3.1 AR Module Overview

Augmented Reality (AR) and Virtual Reality (VR) represent the cutting edge of interactive experience design, encompassing a dynamic range of potentials that engage users in unprecedented ways. The architectural patterns established in the GameCraft 3D desktop project serve as a foundational framework that directly supports development in both AR and VR domains. A comprehensive understanding of Unity's standard 3D environment—specifically, its component model, robust physics system, and versatile C# scripting API—is essential for any developer transitioning into AR or VR development. Mastery of these foundational elements equips creators to build immersive experiences that push the boundaries of interactive technology.

To embark on the AR development pathway leveraging GameCraft's codebase, one must replace the conventional keyboard-and-Rigidbody player movement system with input derived from AR device tracking. This transition is facilitated by Unity's AR Foundation

package, a powerful tool that abstracts the complexities of platform-specific AR SDKs—such as ARCore for Android and ARKit for iOS—behind a unified API. A key component in this setup is the XROrigin, which effectively replaces the standard camera rig, allowing developers to create experiences that align seamlessly with the physical environment. The AR Session component plays a crucial role in managing the lifecycle of the AR experience, handling states such as starting, pausing, and terminating the session as users engage with the augmented world. Moreover, ARRaycastManager streamlines the process of detecting real-world surfaces, substituting the standard Physics.Raycast technology.

Adapting the PlayerMovement.cs script from GameCraft for AR applications is a logical next step, involving a straightforward substitution of Input.GetAxis keyboard readings with inputs from the device's touch screen or motion sensors. This modification allows for fluid, intuitive movement that aligns with the unique interaction models present in augmented reality. Similarly, the collectible coin system, originally developed with trigger-based detection using OnTriggerEnter, operates effectively in AR contexts as well. A virtual coin can be placed accurately on a detected real-world surface, and players can collect it by maneuvering the AR camera within range of its trigger collider, thus activating the existing CoinPickup logic established in GameCraft.

In terms of audio management, AR experiences can leverage the same AudioSource and AudioListener architecture utilized in GameCraft, ensuring consistency and familiarity. By attaching the AudioListener to the AR camera's Transform, developers can achieve a spatial audio experience that corresponds accurately to the user's physical orientation within the environment. The implementation of an AudioManager singleton, utilizing the DontDestroyOnLoad pattern, proves equally beneficial in AR applications, particularly in instances where multiple AR scenes require a shared and persistent audio context.

Furthermore, the GameManager singleton pattern, which effectively manages score tracking across various scenes, directly translates to AR gaming scenarios. In these experiences, maintaining progression and state as players navigate between different real-world locations that activate distinct AR experiences is crucial. The DontDestroyOnLoad approach, successfully used in GameCraft, provides a seamless cross-scene persistence that enhances user engagement in AR multi-scene architectures. Overall, the adaptability of GameCraft's foundational systems to the AR realm facilitates the creation of immersive, interactive experiences that blend the virtual and physical worlds in meaningful ways.

3.2 VR Module Overview

Virtual Reality (VR) development in Unity builds directly upon the foundational elements of 3D scene architecture, the integrated physics system, and the scripting patterns established in GameCraft. Transitioning from traditional desktop 3D development to a VR environment primarily involves substituting the standard camera rig and keyboard input system with the XR Rig, which is provided by the Unity XR Interaction Toolkit (XR ITK) package. This change is critical as it allows developers to leverage VR-specific functionalities.

The XR Origin component in VR serves as a key replacement for GameCraft's Main Camera. It comprises a rig that includes a Camera Offset parent, the VR headset camera, and two hand controller transforms. Unlike the standard camera operations, the follow camera logic (CameraFollow.cs) is not directly applicable in a VR context. This is due to the fact that the headset camera is intrinsically linked to the player's physical head movements, meaning that the camera's position is dictated by the physical device rather than through scripted interpolation. Nevertheless, the concept of maintaining a steady offset between the player's body and the camera rig remains crucial for developing effective VR locomotion systems.

In terms of player movement within the VR environment, the implementation adapts the Rigidbody velocity control from PlayerMovement.cs to work in tandem with the XR controller joystick input. This is achieved through the InputActionReference API of the XR Input Subsystem. The approach for assigning velocity in FixedUpdate, as utilized in GameCraft, remains consistent in VR locomotion scripts. However, the source of the horizontal and vertical input values shifts from the traditional Input.GetAxis to the XR joystick, marking a significant evolution in input handling for VR experiences.

Regarding enemy behaviors, the patrol AI, encapsulated within EnemyPatrol.cs and utilizing the Vector3.MoveTowards method, requires no modification for VR deployment. This is due to the agnostic nature of the patrol script, which can operate seamlessly regardless of whether the player is using a keyboard or VR controller. Similarly, systems like the CoinPickup trigger and the GameManager singleton retain their functionality without change. The primary adjustment necessary for VR interaction lies within the interaction system itself. Instead of requiring players to walk into coins for collection, VR

games implement XR Grab Interactable components, allowing players to physically reach out and grab objects using their hand controllers, enhancing the immersive experience.

Lastly, the user interface (UI) system necessitates a significant redesign for effective VR deployment. Unity's Screen Space Overlay Canvas, which was used in the HUD of GameCraft, is not compatible with VR headsets. Therefore, it is crucial to employ World Space Canvas components, which can be positioned at a fixed point in 3D space or attached to the headset camera's Transform. The underlying logic of the UIManager and GameManager remains unchanged; however, the Canvas's render mode, along with its positioning, requires specific configurations tailored for the VR environment. By aligning these elements effectively, developers can create an intuitive and engaging user interface within virtual reality settings.

CHAPTER 4

PROJECT DEVELOPMENT

This chapter provides a comprehensive and detailed account of the entire process involved in establishing a robust development environment tailored for game design. It guides you step-by-step through the intricacies of creating a new Unity project, meticulously outlining the essential configurations and settings needed to optimize the project for gameplay development. Each aspect of the setup is described with clarity, ensuring that even those new to Unity can follow along seamlessly. Moreover, this chapter delves into the construction of both game levels, offering a thorough walkthrough of the design principles, environmental considerations, and technical requirements necessary to bring your game to life. With ample detail provided at every stage, this guide serves not only as a tutorial but also as a valuable resource, empowering readers to replicate the entire process with confidence and creativity.

4.1 Unity Hub Setup

The development environment setup for the Unity project commenced with the download and installation of Unity Hub 3.x tailored for Windows 64-bit systems. This process began by navigating to the official Unity download page, where the installer was acquired. After downloading, the installer was executed using the default settings, which facilitated a smooth installation experience. Upon successful installation, Unity Hub was launched, prompting the creation of a Unity ID account. This account was established at no cost through the Unity ID portal (id.unity.com). Subsequently, the Unity Personal License was activated by following the steps in Hub's preferences menu—specifically through Preferences > Licenses > Add License > Get a free personal license—thereby confirming eligibility under the guidelines set for personal use.

Once the Unity Hub was fully operational, the installation of Unity Editor 2022.3 LTS was initiated. This was accomplished by selecting the appropriate options under Hub > Installs > Install Editor. In the module configuration step prior to installation, several essential components were chosen to enhance development capabilities. These included Microsoft Visual Studio Community 2022 for C# scripting, Windows Build Support (IL2CPP) to generate standalone Windows executables, and Documentation for offline access to the

Unity API reference. The entire download and installation process was relatively substantial, consuming approximately 5.4 GB of disk space and requiring around 40 minutes on a standard broadband connection. This investment of time and resources set a solid foundation for future development tasks.

Following the installation, an important step was taken to configure Visual Studio IntelliSense, which plays a crucial role in streamlining the coding process. After launching the Unity Editor, the user navigated to Edit > Preferences > External Tools. Here, it was confirmed that the External Script Editor was set to Visual Studio 2022. To facilitate proper integration, the “Regenerate Project Files” option was selected. This action generated the necessary Assembly-CSharp.csproj and Assembly-CSharp-Editor.csproj files, which Visual Studio requires to resolve references from the UnityEngine namespace and provide the indispensable IntelliSense feature. Neglecting this step would have resulted in a frustrating experience for beginner developers, as Visual Studio would fail to offer autocomplete suggestions for Unity classes—a common challenge encountered in the early stages of development.

With the development environment properly set up, the next step involved creating a new Unity 3D project through Hub > New Project. The project template selected was the 3D (Built-In Render Pipeline), and the project was aptly named "GameCraft_3DPlatformer." The decision to select the Built-In Render Pipeline over the Universal Render Pipeline stemmed from its user-friendliness and out-of-the-box functionality. It requires no additional pipeline asset configuration, supports all essential features, including directional lighting, real-time shadows, and Standard Shader materials, making it an ideal choice for newcomers looking to quickly and effectively dive into Unity development.

To further organize the project, four scenes were created and configured within the Build Settings in the specified order: StartScreen (index 0), Level1 (index 1), Level2 (index 2), and GameOver (index 3). Establishing the correct scene build order is vital, as the SceneManager.LoadScene() function throughout the project utilizes these integer indices for seamless navigation between various game states. Moreover, to maintain clarity and efficiency, the project folder structure was meticulously organized into subfolders, including Assets/Scripts, Assets/Scenes, Assets/Materials, Assets/Audio, Assets/Prefabs, and Assets/Fonts. This structured approach not only enhances workflow but also ensures that assets and scripts are easily accessible as development progresses.

4.2 Scene Design

Level 1 was meticulously crafted as an introductory experience, intended to ease players into the game's mechanics and dynamics. To achieve this, the design featured forgiving platform widths of 3 units, paired with a modest elevation range that reached a maximum height of 5 meters. Adding to the challenge, the level included two patrolling enemies positioned on the ground plane, alongside six collectible coins strategically distributed at various heights throughout the environment.

The construction of the scene commenced with a Plane scaled to (5, 1, 5), providing a spacious ground surface measuring 50 x 50 units, which was adorned with a subtle grey material. To enhance visual depth and facilitate an intuitive sense of platform height, a Directional Light was rotated to an angle of (50, -30, 0). This specific orientation helped cast angled shadows, offering players essential cues for navigating the differing elevations in the level.

Strategically placed throughout Level 1, eight platform Cubes were arranged at ascending heights, crafting a non-linear jumping path designed to engage players effectively. Each platform was meticulously scaled to (3, 0.5, 3) and positioned with a vertical separation of approximately 1.25 to 1.5 units between each step. This was carefully calibrated against the player's jumpForce of 7.0 to ensure the jump arc would comfortably clear the height difference, while also providing a slight margin to accommodate varying player skill levels. To ensure visual contrast with the grey ground, a brown-grey material, with an RGB value of (140, 120, 90), was uniformly applied to all platforms.

To establish the player's starting point, the Player capsule was positioned at coordinates (0, 1, 0). It was equipped with a Rigidbody component, with rotation freezing enabled on the X and Z axes, and it also wore a distinctive blue material with an RGB value of (30, 100, 220). The level further included two enemy Cubes, strategically positioned at the start of their patrol paths, characterized by vibrant red materials (RGB 220, 30, 30). These enemies were outfitted with Rigidbody components (marked as Kinematic: true) and given EnemyPatrol scripts, which guided their movements. To enhance interactivity, six Sphere coins, each with a scale of 0.5 and finished with a gold material (RGB 255, 195, 0), featured Is Trigger Sphere Colliders and CoinRotate/CoinPickup scripts to enhance the collection experience, positioned across various collectible locations throughout the level.

Transitioning to Level 2, this segment was developed in a new scene from scratch, with a clear intention of presenting players with a distinctly more challenging environment. The ground plane was enlarged to a scale of (8, 1, 8), resulting in an expansive 80 x 80 unit playfield. To further elevate the challenge, platform widths were narrowed to 1.5 units, and the maximum height was increased to an ambitious 8 meters. This level featured a more intricate non-linear platform arrangement with dead ends that required players to backtrack, enhancing the complexity of navigation.

In Level 2, a total of four enemies were introduced, two of which patrolled the ground, while the other two occupied elevated platforms at varying heights. To spice up player engagement, a rich distribution of ten coins was applied throughout the level, including challenging placements atop elevated platforms and on moving platform surfaces.

Moreover, two moving platform Cubes were integrated into the design of Level 2, positioned in ways that gate access to high-value coin clusters, demanding strategic timing from players. Each moving platform was carefully designed with two child empty GameObjects, named PointA and PointB, located 6 to 8 units apart horizontally. These were assigned as Transform references in the MovingPlatform script to dictate their movement. One platform operates at a gentle speed of 0.8 units per second for journey fraction changes, while the other platform operates at a swifter speed of 1.2 units per second, creating a diverse range of movement rhythms. This variance not only adds challenge but also requires players to gauge their timing accurately to succeed in collecting the coveted coins.

4.3 Prefab and Asset Configuration

Prefabs are a central feature of Unity's asset management system, serving as a powerful tool for developers to create reusable templates for GameObjects. Essentially, a Prefab allows a specific GameObject to be instantiated multiple times across a scene or even across various scenes while ensuring that all instances share the same configuration of components. This consistency is key in game development, as it streamlines the process of updating and maintaining shared objects. When modifications are made to the Prefab asset itself, those changes are automatically propagated to all instances wherever they exist, providing developers with a single point of maintenance. This not only enhances efficiency but also minimizes errors, ensuring that the gameplay experience remains uniform.

In the project discussed, three distinct Prefab assets were carefully created and stored in the Assets/Prefabs/ directory. The Enemy Prefab is particularly noteworthy as it consolidates the comprehensive configuration necessary for enemy characters. This Prefab includes a parent Cube GameObject featuring a vibrant red material, equipped with a box collider, a kinematic Rigidbody, and an EnemyPatrol script to control its movements. Additionally, it incorporates a child object, the TopTrigger Cube, complete with its own trigger collider and an EnemyTop script, along with two essential child GameObjects designated as WaypointA and WaypointB. By utilizing this Prefab for creating enemies in Level 2, the project ensured that the four enemies in this level had the same component configuration as those in Level 1. The only variations came from the patrol waypoint positions, which were set individually per instance, making the design process both efficient and consistent.

Another vital Prefab created for this project is the Coin Prefab. This Prefab captures the attributes of the collectible gold Sphere, meticulously designed with a scaled transform of (0.5, 0.5, 0.5) for uniformity. It features a gold material that gives it a shiny appearance, along with a trigger Sphere Collider to facilitate player interaction. Additionally, scripts such as CoinRotate and CoinPickup have been integrated into this Prefab. Across both levels of the game—six coins in Level 1 and ten in Level 2—a total of sixteen coins were instantiated from this single Prefab. This strategy ensured that crucial settings, such as the Is Trigger option on the Sphere Collider—which relates to a notable bug discussed in Section 6.1.5—were established correctly for all coin instances simply by modifying the Prefab once, instead of adjusting each individual coin separately.

The third essential Prefab created was the Finish Platform Prefab, designed specifically for the final elevated platform in each level, where the last required coins are located. This Prefab stands out as it employs a distinct grey-green material, subtly indicating its importance to the player. To enhance its visibility and desirability, a Particle System component has been integrated, set to emit a gentle sparkling effect that draws the player's attention toward the level's completion objective. This thoughtful design choice not only provides visual feedback but also guides players in their journey towards completing the game.

In addition to the Prefabs, a variety of material assets were created and organized within the Assets/Materials/ directory to represent each distinct surface type encountered in the game. The materials include GroundMaterial, characterized by a grey color (RGB 180, 180, 180), and PlatformMaterial, which carries a brown-grey hue (RGB 140, 120, 90). The

PlayerMaterial stands out with its vibrant blue shade (RGB 30, 100, 220), while EnemyMaterial is marked by its striking red color (RGB 220, 30, 30). The CoinMaterial reflects a luxurious gold tone with a Metallic setting of 0.8 and Smoothness of 0.9, applied using Unity's Standard Shader, ensuring a reflective quality appropriate for collectible items. Lastly, the FinishMaterial employs the same grey-green color as the platform, with RGB values of 100, 160, 100, effectively tying the visual elements together. Most materials utilize Unity's Standard Shader with default settings, except for CoinMaterial, which leverages the metallic workflow to achieve its distinctive golden sheen. This cohesive approach to materials is crucial in creating an immersive and visually appealing gaming environment.

CHAPTER 5

IMPLEMENTATION

This chapter offers an exhaustive technical overview of each C# script integrated into the GameCraft project, presenting a detailed account of their functionalities and roles within the broader context of the game. Each script is meticulously documented to include not only its core responsibilities but also the various Unity lifecycle methods it employs, providing insights into how these methods interact with the game's architecture. Furthermore, the chapter elaborates on the serialized Inspector fields associated with each script, highlighting how these fields enhance the user experience and streamline the development process. In addition to these technical aspects, the narrative delves into the specific implementation decisions that were made during the development, articulating the rationale behind selecting particular approaches over potential alternatives. This comprehensive analysis aims to furnish readers with a robust understanding of the scripting landscape within GameCraft, thereby empowering them to grasp the intricacies of the project's codebase and the thoughtful considerations that guided its development.

5.1 AR Implementation

The AR implementation module of the project encompasses the scripting architecture that underpins essential components such as player movement, camera follow mechanics, and interaction systems, all of which collectively deliver a rich gameplay experience in augmented reality (AR). These scripts are designed with scalability in mind, ensuring that core systems like physics-based movement, visual tracking, and game state management can seamlessly transition into AR deployment.

At the heart of the gameplay mechanics is the `PlayerMovement.cs` script, which is affixed to the Player capsule GameObject. This script defines four serialized fields that govern player control: a `float moveSpeed`, defaulting at 5.0, which regulates the magnitude of horizontal movement; a `float jumpForce`, set to 7.0, that dictates the upward impulse applied during jumps; a `LayerMask groundLayer`, which specifies the various physics layers recognized as ground; and a `Transform groundCheck`, which references an empty child GameObject situated at the base of the capsule, serving as the origin for grounding detection.

During the `Start()` method, the script leverages a crucial performance optimization by calling `GetComponent<Rigidbody>()` and storing its result in a private field called `rb`. This caching is a pivotal practice in Unity development, as it avoids the performance overhead associated with repeated calls to `GetComponent<T>()` in every frame, which entails searching through the component list. Instead, by executing this call once during the `Start()`, the script efficiently reuses the result during each frame update.

In the `Update()` method, player input is read through `Input.GetAxis("Horizontal")` and `Input.GetAxis("Vertical")` for movement, alongside the `Input.GetKeyDown(KeyCode.Space)` check for jump detection. Concurrently, the `isGrounded` boolean is updated each frame using `Physics.CheckSphere(groundCheck.position, 0.2f, groundLayer)`, allowing for real-time grounding detection to ensure the player can only jump when on solid ground.

Movement is applied within the `FixedUpdate()` method to synchronize with Unity's physics simulation timestep, ensuring smooth interactions with the game's environment. The target velocity is set by assigning `rb.velocity = new Vector3(horizontal * moveSpeed, rb.velocity.y, vertical * moveSpeed)`. A significant design choice is made here to preserve the player's existing Y component of velocity (`rb.velocity.y`), instead of zeroing it out. By maintaining the Y velocity, gravity can exert its effect continuously, facilitating realistic falling mechanics while player control is limited to horizontal movement.

Jumping mechanics are uniquely handled in the `Update()` method rather than `FixedUpdate()`, primarily because `Input.GetKeyDown()` is designed to return `true` for only a single frame. This design consideration is crucial, as it may not align perfectly with the FixedUpdate cycle. The jump is implemented using `rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse)`, resulting in an instantaneous change in vertical velocity by `jumpForce` meters per second in that specific physics step. To enforce proper gameplay mechanics, the `isGrounded` guard ensures that the jump force is applied at most once during any grounded frame.

For the camera mechanics, the `CameraFollow.cs` script is attached to the Main Camera. In the `Start()` method, it calculates the offset from the player by utilizing `transform.position - player.position`. During each frame, in `LateUpdate()`, the script computes the desired position for the camera as `player.position + offset`, subsequently using `Vector3.Lerp(transform.position, desiredPosition, smoothSpeed)` to achieve a smooth transition. The `smoothSpeed`, set to 0.125, ensures that the camera moves 12.5%

of the distance to the target position each frame, resulting in a gradual and smooth convergence. The use of `LateUpdate()` guarantees that the player's position has already been updated before the camera adjusts its own position, thereby preventing lag or jitter in camera movement.

Lastly, the `MovingPlatform.cs` script introduces a dynamic element to the gameplay by managing the movement of platforms between two points, referred to as pointA and pointB. A `journeyFraction` float is employed, which increments from 0 to 1 as the platform travels from pointA to pointB, then decrements back to 0 as it reverses direction. Each frame within the `FixedUpdate()` method, the fraction is adjusted by `platformSpeed * Time.deltaTime`. The actual platform position is set through `transform.position = Vector3.Lerp(pointA.position, pointB.position, journeyFraction)`. When the fraction hits either 1 or 0, the `movingToB` boolean is toggled to reverse the direction of the platform. Furthermore, collision detection is implemented to impact the player's traversal; upon collision with the player, the script sets the player's `Transform.parent` to `this.transform`, parenting them to the moving platform, and restores the player's parent to null in `OnCollisionExit()` to return them to world-space independence.

By meticulously structuring these scripts, the AR implementation not only promotes an engaging gameplay experience but also lays a solid foundation for further scalability and versatility in augmented reality contexts. Each element of the script interacts cohesively, fostering a sophisticated environment where players can navigate, interact, and enjoy the immersive qualities of AR gaming.

5.2 VR Implementation

The VR implementation module encompasses several critical components, including the enemy AI system, coin collection mechanics, and audio management. These systems govern non-player game entities and manage how players interact with the game environment. Each of these features plays an essential role in creating an engaging and dynamic gameplay experience.

At the core of enemy behavior, the script EnemyPatrol.cs is assigned to every enemy Cube in the game. This script initializes two Transform fields, waypointA and waypointB, which are designated in the Unity Inspector. Additionally, it uses a float variable, patrolSpeed, defaulting at 2.0 for consistent movement. Within the Update()

method, the target for the enemy's movement is determined based on the boolean variable movingToB. If this variable is true, the target is set to waypointB.position; otherwise, it refers to waypointA.position. The actual movement is handled by the line transform.position = Vector3.MoveTowards(transform.position, target, patrolSpeed * Time.deltaTime). The use of Vector3.MoveTowards rather than Lerp is crucial, as it allows for fixed movement speed measured in meters per second, ensuring the enemy navigates the space consistently, regardless of the distance to its waypoints. Moreover, the distance check implemented with Vector3.Distance(transform.position, target) < 0.05f triggers a change in direction, enhancing the enemy's patrol dynamics.

To enhance player interaction with enemies, the EnemyTop.cs script is employed on a specially configured flat Cube that serves as a Trigger collider, strategically placed on the enemy's top face. The script utilizes OnTriggerEnter(Collider other) to detect when the player collides with this trigger area. If the player is detected and their Rigidbody's Y velocity is below -0.1f (indicating a downward attack), several actions are initiated. The script calls GetComponentInParent<Enemy>().Die(), which subsequently plays a stomp sound via AudioManager.PlaySFX(stompClip). It also increments the player's score by invoking GameManager.instance.AddScore(10), applies a small upward impulse to the player's Rigidbody for an enjoyable bounce effect, and finally removes the enemy from the game scene with Destroy(gameObject).

On the player side, the PlayerHealth.cs script orchestrates the sequence that follows when a player encounters an enemy. It utilizes OnCollisionEnter(Collision collision) to listen for collisions with any game object tagged as "Enemy." Upon detection, the script commences a coroutine that first stops any currently playing music with AudioManager.instance.StopMusic(), followed by the playback of a game-over sound effect through AudioManager.PlaySFX(gameOverClip). The coroutine includes a wait period with yield return new WaitForSeconds(0.5f) to ensure the sound effect concludes before transitioning to the game over screen by calling SceneManager.LoadScene("GameOver"). This coroutine pattern is vital because invoking SceneManager.LoadScene() in the same frame as the audio trigger would lead to the premature unloading of the audio system, preventing the sound from being fully played.

Coin collection is managed by the CoinPickup.cs script, which leverages OnTriggerEnter(Collider other) to handle player interactions upon collecting a coin. The script confirms the collision with the player using other.CompareTag("Player"). Once verification is complete, it executes AudioManager.PlaySFX(coinClip) to play the coin

collection sound, calls GameManager.instance.AddCoin() to update the coin count, and finishes the process with Destroy(gameObject) to remove the coin from the scene. The GameManager.AddCoin() method increments both the total number of coins collected and the player's score. It also updates the on-screen display by invoking UIManager.instance.UpdateHUD(), and subsequently checks whether the level is complete by calling CheckLevelComplete() to determine if the total coins collected meets or exceeds the target. If the player meets the criteria, the script triggers UIManager.instance.ShowLevelCompleteScreen() to show the level completion overlay, providing players with feedback on their success.

Lastly, the AudioManager.cs script serves as a persistent singleton that manages all audio-related functionalities in the game. It is attached to a dedicated empty GameObject named AudioManager. The singleton pattern is implemented through a static Instance property, ensuring that if Instance is null, it assigns Instance = this and calls DontDestroyOnLoad(gameObject) to maintain its presence across different scenes. To prevent duplicate AudioManager instances when a scene is reloaded, the script includes logic to destroy any existing AudioManager objects. The AudioManager features two AudioSource components: musicSource (configured for looping, with a spatial blend of 0 and a volume level of 0.5) and sfxSource (set to non-looping with a spatial blend of 0 as well). The public static method PlaySFX(AudioClip clip) allows for multiple simultaneous sound effects through sfxSource.PlayOneShot(clip), ensuring a rich auditory experience without hindering performance.

Together, these scripts and systems create a cohesive and immersive gameplay environment where player interactions, enemy behaviors, and audio management work seamlessly to enhance the overall gaming experience.

5.3 C# Scripting

The C# scripting module focuses on essential components such as game state management, user interface (UI), and score systems, intertwining various gameplay elements to create a cohesive game loop. At the heart of this architecture is the GameManager.cs, which serves as the central game state manager. Implemented as a persistent singleton using DontDestroyOnLoad, the GameManager ensures that important game states are preserved between scene transitions. Within this script, several private variables are maintained: an integer score representing the total accumulated score across all levels, an integer

coinsCollected to track the coins gathered in the current level, an integer totalCoins set by LevelManager upon scene load (indicating the total coins available in the current level), and currentLevel, an integer that reflects the current scene's build index, primarily used for UI display purposes.

The GameManager offers several public methods to facilitate interaction with other scripts. For instance, AddScore(int amount) increments the current score by a specified amount and triggers a call to UIManager.instance.UpdateScoreDisplay(score) to refresh the displayed score in the UI. The method AddCoin() plays a vital role in managing coin collection: it increments coinsCollected, invokes AddScore(5) to reward the player for collecting a coin, updates the UI with UIManager.instance.UpdateCoinDisplay(coinsCollected, totalCoins), and finally checks if the level is complete by calling CheckLevelComplete(). Additionally, the ResetGame() method zeros all relevant variables and is called before loading the Retry scene, effectively resetting the game state for a fresh start.

To handle the user interface, UIManager.cs is utilized as a per-scene script, attached to the Canvas of each scene. This script maintains serialized references to all UI elements present in the scene, including TMP_Text objects for scoreText and coinText (used in the HUD), GameObjects for levelCompletePanel and gameOverPanel, and Buttons for nextLevelButton and retryButton. The method UpdateScoreDisplay(int score) dynamically updates the score display by setting scoreText.text to reflect the current score with the format $"Score: {score}". Similarly, UpdateCoinDisplay(int collected, int total) updates the coin display to show the collected and total coins with the format $"Coins: {collected}/{total}". When a level is completed, ShowLevelCompleteScreen() is executed to activate the level completion panel and display the final score. Button callbacks are assigned directly in the Inspector, linking actions like OnPlayButtonClicked to load the first level, OnNextLevelButtonClicked to advance to the next level, OnRetryButtonClicked to reset the game state and reload the first level, and OnMainMenuButtonClicked to return to the main menu.

Coin interactions are managed by the CoinRotate.cs script, which is attached to each coin object. This script executes a rotation of the coin along its Y-axis by calling transform.Rotate(0f, rotationSpeed * Time.deltaTime, 0f) within the Update() method. The rotationSpeed is a serialized float initialized to 90 degrees per second. This approach creates a visually appealing continuous rotation, indicating to players that the coin is collectible. Importantly, the rotation is applied directly to the local Transform rather than

utilizing a Rigidbody, as applying torque forces to surfaces through a Rigidbody would complicate the interaction dynamics.

Lastly, LevelManager.cs serves as another per-scene script that enhances the game's efficiency in terms of coin management. Upon the Start event of each scene, it counts the number of CoinPickup components present by using FindObjectsOfType<CoinPickup>().Length and assigns this value to GameManager.instance.totalCoins. This automatic counting not only removes the need for manual updates to the total coin count in the GameManager whenever coins are added or removed, but it also significantly speeds up the level design iteration process, providing both safety and convenience. By limiting FindObjectsOfType to the Start method (rather than repeatedly within Update), the script mitigates performance costs associated with large scenes, ensuring that the counting overhead is incurred only once during scene load.

Overall, this comprehensive architecture provides a solid foundation for managing gameplay elements effectively, thereby enhancing player experience and facilitating streamlined development processes.

CHAPTER 6

DEBUGGING

Debugging is a crucial and intellectually demanding phase of software development, particularly in the realm of interactive real-time systems such as video games, where the complexity can significantly increase due to a variety of factors. In these environments, bugs can emerge from an intricate interplay of elements, including the timing of physics simulations, dependencies arising from execution order, errors in component configuration, and behaviors that fluctuate with changes in frame rates. Each of these factors presents unique challenges that require careful analysis and resolution. This chapter meticulously documents all notable bugs encountered throughout the development process of GameCraft, providing a comprehensive root cause analysis for each issue. Additionally, it details the specific fixes that were implemented to resolve these problems and outlines the rigorous verification methods employed to confirm that the resolutions were effective. Through this thorough examination, the chapter aims to offer valuable insights into the debugging process, highlighting the complexities and solutions involved in creating a robust interactive experience.

6.1 Common Errors and Fixes

Bug 1 — After installing Unity and Visual Studio, I encountered an issue where IntelliSense was not working; specifically, when opening a C# script from the Project window, no suggestions appeared for Unity-specific types like MonoBehaviour, Rigidbody, or Vector3, and pressing the dot after 'transform' yielded no property suggestions. The root cause was that Unity had not generated the necessary .csproj project files for Visual Studio to recognize the UnityEngine assembly, causing Visual Studio to treat the Unity project as a standard C# file without proper assembly context. To resolve this, I opened the Unity Editor, navigated to Edit > Preferences > External Tools, ensured that the External Script Editor was set to Microsoft Visual Studio 2022, and clicked on Regenerate Project Files. After closing and reopening the script file, IntelliSense became fully functional, which I verified by typing 'GetComponent<' in Visual Studio, resulting in an autocomplete list of all available Unity component types.

Bug 2 — The player experienced sliding after releasing movement keys, causing the game to feel unresponsive and imprecise, as the player capsule continued to move in the last direction for one to two seconds. This issue stemmed from the initial movement implementation using rb.AddForce(direction * moveSpeed, ForceMode.Force) in FixedUpdate, which accumulated velocity without applying deceleration due to the Rigidbody's linear Drag being set to 0 by default. To resolve this, we replaced AddForce with direct velocity assignment: rb.velocity = new Vector3(horizontal * moveSpeed, rb.velocity.y, vertical * moveSpeed), allowing for an immediate stop by resetting horizontal velocity to zero when there was no input. Additionally, we increased the Rigidbody Drag to 5 as a safeguard against any residual momentum. Verification showed that pressing and releasing the WASD keys at maximum speed resulted in an immediate stop without any observable sliding.

Bug 3 — When the player character fell from a high platform (approximately 6 meters above ground) while the game was running at a low frame rate (below 30 FPS on an underpowered test machine), it passed through the ground plane and fell out of the level. The root cause was identified as the Rigidbody's Collision Detection being set to Discrete (the default setting), which only checks for collisions at the end of each physics timestep. At 0.02-second timesteps, a fast-falling object can move far enough in a single step to completely bypass detection of a thin plane. To resolve this issue, the Collision Detection was changed from Discrete to Continuous in the Rigidbody Inspector. Continuous mode conducts additional sub-step collision checks for fast-moving rigid bodies. Verification of the fix involved repeatedly dropping the player from the maximum level height of 8 meters at low frame rates, which confirmed that no tunneling occurred.

Bug 4 — The enemy stomp was not triggering as expected when jumping directly on top of an enemy, resulting in a physics bounce without any enemy death, sound, or score award. The root cause was twofold: first, the Player GameObject lacked the Player tag, defaulting to Untagged, which caused the CompareTag("Player") check in EnemyTop.OnTriggerEnter to consistently return false. Second, the TopTrigger's child Cube had a Y scale of only 0.1 units, creating an excessively thin detection zone that failed to capture approaching capsules from above. To resolve these issues, the Player tag was applied via the tag dropdown in the Inspector, and the TopTrigger's Y scale was increased from 0.1 to 0.3 units. Additionally, a temporary Debug.Log statement was added to OnTriggerEnter to confirm the trigger activation, which was subsequently removed after

successful testing. Verification was achieved by jumping on each enemy in Levels 1 and 2, where stomp deaths were reliably triggered on ten consecutive attempts.

Bug 5 — In the game, players were unable to collect coins as they walked into coin spheres, with no sound playing, score incrementing, or visual feedback of coins being collected, resulting in them remaining in the scene. The root cause was identified as the Sphere Collider on each coin having the "Is Trigger" option unchecked, which prevents the OnTriggerEnter() method in CoinPickup.cs from being called, as solid colliders do not register trigger events. Consequently, players were actually pushed aside by the physics of the solid colliders, a phenomenon that was not visually evident due to the small size of the coins. To resolve this issue, all coin GameObjects were selected in the Hierarchy, the Sphere Collider was modified in the Inspector to enable the "Is Trigger" checkbox, and this change was applied simultaneously to all coins through multi-object editing. Additionally, the Coin Prefab was updated to ensure that future instances would have the "Is Trigger" option enabled by default. Verification showed that when walking through each of the six Level 1 coins, players successfully triggered the collection sound, saw the score increment, and noticed an immediate removal of the coins from the scene.

Bug 6 — The follow camera experienced a noticeable single-frame jitter or stutter during diagonal movement, particularly evident at high speeds. This issue stemmed from the camera position update being placed in the Update() method, which executes in an order that isn't strictly defined among different scripts. Consequently, the camera's Update could sometimes run before the player's Update—resulting in a lag that captured the previous frame's position—and at other times afterward, leading to no lag. This inconsistency created the perceptible jitter. To resolve the problem, the camera position update was moved from Update() to LateUpdate(), ensuring that it always executes after all Update() calls in the same frame. As a result, the camera consistently reads the player's final position post-movement for each frame. Verification tests, involving running the player diagonally at full speed for 30 seconds, showed no observable camera jitter.

6.2 Testing on Device

Beyond merely addressing individual bug fixes, a comprehensive approach to device-level testing was undertaken across various hardware configurations. This was essential to ensure that the game functioned correctly outside the familiar environment of the development

machine's Unity Editor. To achieve this, three distinct testing configurations were employed: the Unity Editor Play Mode on the development machine, which features an Intel Core i7 processor, 16 GB of RAM, and an NVIDIA GTX 1650 graphics card; Play Mode on a budget student laptop equipped with an Intel Core i5 processor, 8 GB of RAM, and Intel UHD integrated graphics; and finally, the standalone Windows build executable tested on the development machine itself.

During the testing phase on the budget laptop, two additional bugs were identified that had previously gone unnoticed. The first issue involved the Moving Platform's player parenting system, which occasionally failed to properly parent the player character when the collision velocity was exceedingly low. This usually occurred when the player slowly walked onto the platform instead of jumping onto it. After close inspection, we determined that this problem could be resolved by removing an unintended velocity magnitude guard that had been added to the OnCollisionEnter function.

The second issue related to the background music AudioSource, which sometimes exhibited crackling artifacts. This distortion was particularly noticeable when the integrated audio hardware was in use. To mitigate this problem, we increased the DSP Buffer Size in the Project Settings under the Audio category, switching from the default 'Best Latency' option to 'Best Performance.' Although this adjustment comes at the cost of slightly increased audio latency, it ultimately provided a more stable audio output, enhancing the overall gameplay experience.

Moving on to the standalone Windows build, this was created through a straightforward process involving the selection of Windows (x86_64) as the target platform within the File > Build Settings menu, followed by the 'Build and Run' option. The build process itself took approximately 3 minutes and 48 seconds to complete, resulting in the production of a 74.2 MB folder containing both the executable file and the accompanying data directory. Upon launching the game from the executable, it was confirmed that everything functioned smoothly: all scenes loaded without any errors, and all game systems operated in a manner indistinguishable from the Unity Editor experience. This successful build validated that there were no dependencies on Editor-only APIs in the production code, an important consideration for a stable release.

Performance profiling conducted using the Unity Profiler window in Play Mode provided valuable insights into the game’s performance metrics. On the development machine, the average frame rate was an impressive 94 FPS, whereas the budget laptop recorded a more

modest 47 FPS. Further breakdown of the performance data indicated that the average CPU frame budget on the budget laptop was approximately 21.3 ms. Interestingly, the rendering pass, which included DrawMesh calls for platform geometry, accounted for 38% of this budget, while physics operations—comprising FixedUpdate and contact detection—made up 29%. Notably, no garbage collection events were observed during gameplay on either machine, which confirmed that both the Update and FixedUpdate loops were efficiently allocating no heap memory throughout the runtime. This optimization is a testament to the careful coding practices implemented during the game's development.

CHAPTER 7

TESTING

This chapter provides a comprehensive overview of the testing strategy employed for the GameCraft project, detailing the systematic approach taken to ensure the quality and functionality of the game. It encompasses a variety of testing methodologies, including rigorous unit testing of individual game systems to verify their performance and reliability. Furthermore, it discusses the integration testing conducted to assess the interactions and communication between different systems, ensuring they work harmoniously together. In addition, user acceptance testing was carried out with independent evaluators to gather feedback on the user experience and overall satisfaction. It is important to note that all testing processes were meticulously executed within the Unity Editor Play Mode, as well as within a standalone Windows build executable, allowing us to simulate real-world gameplay scenarios and thoroughly evaluate the game's performance across different environments.

7.1 Unit Testing

Unit tests for GameCraft were carried out in carefully designed test scenes that focused exclusively on the system under evaluation, thereby ensuring complete isolation from all other game systems. This deliberate separation was crucial, as it enabled the verification of each system's behavior independently, allowing for a more accurate and thorough assessment before moving on to integration testing. The results of these unit tests were meticulously compared against established expected outcomes, which were documented in detail to cover a range of scenarios, including both typical inputs and challenging boundary conditions. This structured approach not only facilitated early identification of potential issues but also laid a solid foundation for a more robust and seamless integration process later in the development cycle.

Test 1.1 — In a minimal test scene featuring only a ground plane and the player character, I conducted a thorough examination of player movement direction by pressing each of the eight possible directional inputs: W, A, S, D, WA, WD, SA, and SD. The goal of this test was to assess whether the player's movement corresponded accurately to each

specific input without any unintended perpendicular drift. I anticipated that the player would move directly in the indicated direction corresponding to the keys pressed. Upon observation, I found that all eight directional inputs successfully resulted in the expected movement behavior, demonstrating precise control and responsiveness. The results confirmed that the movement mechanics were functioning correctly, leading to a successful outcome for this test scenario. Overall, this indicates a robust implementation of player movement controls. Pass.

Test 1.2 — Jump Grounding Constraint. Method: Pressed Space bar while standing on ground. Pressed Space bar again while airborne (three attempts). Pressed Space bar immediately upon landing (five attempts). Expected: Jump fires on ground press; no jump while airborne; jump fires correctly on landing. Actual: All conditions behaved as expected across all attempts. Pass.

Test 1.3 — The evaluation of Physics Collision Integrity was conducted by dropping the player from the maximum height level of 8 meters onto various ground surfaces, including the ground plane and all types of platforms, with a total of twenty drops on each surface. The expected outcome was straightforward: the player should land safely on all surfaces without experiencing any falls through the ground or platforms. Upon completing the test, it was confirmed that all twenty drops onto the ground plane, as well as the twenty drops onto the platforms, resulted in successful landings without any issues or glitches. Therefore, the result of the test is a clear pass, indicating that the collision detection system is functioning as intended across all tested surfaces.

Test 1.4 — In evaluating the consistency of enemy patrol behavior, I conducted an observation of a single enemy character assigned with waypoints spaced 10 meters apart. This observation spanned a duration of 60 seconds, during which I noted approximately 24 complete patrol cycles. The expected outcome was that the enemy would consistently maintain a speed of 2.0 units per second throughout the patrol, executing clean reversals at each waypoint and exhibiting no signs of positional drift. Upon analyzing the actual performance, I found that the enemy successfully completed all 24 cycles without any observable variations in speed or drift. Based on these findings, the patrol behavior is deemed to have passed the consistency test.

Test 1.5 — In the assessment of Coin Collection Completeness, I employed a detailed methodology by carefully collecting each of the 16 coins from both levels during a dedicated test playthrough. The expected outcome for this process included the activation

of a collection sound for each coin, the visual disappearance of the coin upon collection, an increment in the player's score by 5 points, and an immediate update of the HUD coin count to reflect the collection. Upon executing the test, I successfully collected all 16 coins, and I observed that the game provided the correct audio cues, appropriate visual feedback, and accurate score responses for each coin collected. This indicates that the collection mechanics are functioning as intended. Therefore, the results of the test are deemed a pass.

Test 1.6 — The Enemy Stomp method was extensively tested by executing a standing jump directly above each enemy's center across both levels, with a total of ten attempts per enemy, encompassing 28 unique enemies throughout the testing phases. The expected outcome was a 100% stomp success rate, which would trigger the distinctive death sound, ensure the enemy's disappearance, and award the player with a score of +10 for each successful stomp. However, the actual results revealed 27 out of 28 successful stomps. Notably, there was one instance of failure that occurred when the player inadvertently jumped from a position directly beside the enemy, resulting in contact with the side collider first rather than the center. This scenario was identified as a valid side-kill detection, representing an edge case rather than a flaw in the game's mechanics. Therefore, while the testing largely confirmed the effectiveness of the stomping method, this particular edge case has been documented for reference, ultimately leading to a pass for the feature with this isolated incident noted for future consideration..

Test 1.7 — The Level Completion Trigger was assessed by collecting the final coin in both Level 1 and Level 2 a total of ten times each. The expected outcome was that the Level Complete screen would appear immediately following the collection of the final coin, displaying the correct total score. Upon thorough testing, it was confirmed that the Level Complete screen appeared accurately and promptly after each of the 20 final coin collections, matching the expected results without any discrepancies. This consistent performance indicates a successful pass for the test.

Test 1.8 — The AudioManager's Singleton Persistence was thoroughly tested by playing through the StartScreen and transitioning to Level 1 and then to Level 2 a total of ten times. The expected outcome was that the background music would play continuously, without any interruptions, halts, or restarts during the scene transitions. To assess this functionality, we conducted a comprehensive check across all ten complete playthroughs. Remarkably, the actual results confirmed that the music remained uninterrupted throughout

each transition, seamlessly providing a consistent audio experience. Based on these observations, we can confidently conclude that the feature has passed successfully.

7.2 Integration Testing

Integration tests were conducted to thoroughly verify the correct behavior of various systems functioning together seamlessly. To achieve this, six distinct integration test scenarios were carefully defined, each specifically targeting a particular system boundary. These scenarios ensured that the interactions between different components not only met the specified requirements but also highlighted any potential issues that might arise when systems interfaced with one another. Through this comprehensive testing approach, we aimed to ensure reliability and stability in the overall system performance.

Integration Test 2.1 — The Level 1 Game Loop has been successfully completed, providing a seamless experience from the Start Screen through to the Level Complete screen. During testing, it was confirmed that clicking the Play button effectively loads Level 1, with the player spawning at coordinates (0,1,0). The Heads-Up Display (HUD) correctly displays the initial score as 0 and the number of coins collected as 0 out of a total of 6. All 6 collectible coins within the level are functional and can be gathered without any issues. Additionally, defeating an enemy by performing a stomp action successfully increments the player's score, enhancing the gameplay experience. Upon completing the level, the Level Complete screen accurately showcases the total score accrued after all coins have been collected. Transitioning to the Next Level smoothly loads Level 2, with the player's score being carried forward as intended. Overall, all verifications have passed without any discrepancies. Result: All criteria met, therefore deemed a pass.

Integration Test 2.2 — In the initial testing phase of the game, specifically Level 1, I encountered a scenario where the player walked directly into an enemy's territory. Upon doing so, I verified several key functionalities: the death sound effect played appropriately, instantly capturing the moment of defeat; the background music seamlessly halted, creating a stark contrast that accentuated the Game Over experience; after a brief delay of 0.5 seconds, the Game Over scene successfully loaded, providing players with immediate feedback on their failure. Notably, the Game Over screen displayed the score at the precise moment of death, ensuring players could gauge their performance right up until the end. To further assess the game's response, I cycled through the death and retry process five

times in total. In each case, the system consistently performed as expected, with all five cycles executing correctly without any glitches or discrepancies. Overall, this series of tests resulted in a clear pass, confirming that the death-retry loop is functioning as intended.

Integration Test 2.3 — Test Case for Score Persistence When Moving Between Levels. Score obtained on Level 1: 40 (eight coins multiplied by five, without stomping any enemy). Moved to Level 2. Confirmed HUD showing Score: 40 on entering Level 2. Picked up two coins on Level 2. Confirmed that score was changed to 50. Moved from Level 2 to Level Complete Screen. Confirmed final score is 50. Pass.

Integration Test 2.4 — Moving Platform + Player Collecting. In level 2, was able to ride moving platform through its entire journey four times. During ride, picked up coin on the moving platform. Confirmed that player traveled along with the platform without slipping, collected coin successfully with score update, and exited the platform gracefully after stepping off its edge. Passed.

Integration Test 2.5 — Concurrent Audio Events. In Level 1, quickly got three coins in succession (1 second). Confirmed that all three sounds for coin acquisition played concurrently without the sound getting cut off. Jumped while acquiring the coins. Confirmed that sound for jumping played concurrently with the sounds for coins. Stomped on the enemy right after collecting the coin. Confirmed that sound for stomping plays concurrently with the still-playing coin sound.

Integration Test 2.6 — Main Menu Return. From the Game Over screen at Level 2, clicked on Main Menu. Confirmed that StartScreen was loaded without any leftover game status (score not shown in the StartScreen, proving GameManager was reset). Pass.

7.3 User Acceptance Testing

User acceptance testing was performed using four unbiased testers, fellow students from the Computer Science and Business Systems course who were unfamiliar with the GameCraft assignment. These testers were provided with an executable of the Windows build and requested to play both levels without any assistance or guidance.

Evaluator 1 (Game experience: None). Passed level 1 for the third time. Passed level 2 for the fifth time. Comments: "The controls are very responsive. I did not know that I

could stomp on the enemy characters initially, maybe you should put a hint there. It is very easy to notice coins since they rotate." Usability Problems found: No legend for controls.

Evaluator 2 (Game experience: Moderate). First try on Level 1, Second try on Level 2. Comments: 'Jumping and movement feels natural, like in old-school platform games. The timing of the moving platforms is well done. It would be nice if there were some way to tell how many more levels I have left.' Usability problem found: Lack of level status information.

Evaluator 3 (Game experience: None). Passed level one on second try. Not able to pass level two in time limit (ten tries). Comments: "The narrow platforms in level two are extremely difficult; perhaps even too difficult for a beginner. Sound effects were nice." Usability Problems: Sudden increase in difficulty at level two may be too much for novices.

Evaluator 4 (Game experience: High). Successfully completed both levels in the first try. Comments received: "Very good basic platform game mechanics. The physics of the game seem quite good. Stomping enemies feels very good. Speed timer would be nice to have, as well as a checkpoint system." Usability problems encountered: Checkpoint system is lacking.

Summary of results from the User Acceptance Testing: The testers were able to complete Level 1 without fail; 3 out of 4 testers managed to complete Level 2 during the test period. All testers felt that the controls are easy enough to use without needing any form of instruction. Feedback included: desire for a legend of controls at the beginning of each level, death reset mechanism halfway through levels, and progress bar indicator for the level.

CHAPTER 8

OUTPUT

This chapter includes all the outputs recorded throughout the entire GameCraft project, starting from the moment the application is opened and running through until the end of Level 2.

8.1 Start Screen Output

Scene Index 0 - The Start Screen is displayed right after the initialization process. It is drawn on top of a static camera scene showing the Level 1 map using the Screen Space Overlay Canvas component. The title of the game GAMECRAFT is written using a large and bold font, centered horizontally at the top third part of the screen. The subtitle Build Your Own Adventure is placed beneath the main title using an italicized font style. There is a rectangle-shaped Play button with white text that is centered vertically at the bottom half of the screen. Upon hovering, the color changes to light grey.

8.2 Level 1 Gameplay Output

Output 1 — Player Spawn. In Level 1, the player blue capsule starts from the world coordinates (0, 1, 0). The camera is positioned at about (0, 5, -8), giving a pleasant third-person view with the player's character seen slightly from above and in front. The HUD in the upper left corner shows 'Score: 0' and 'Coins: 0/6', using TextMeshPro font in white against a translucent black background panel.

Output 2 — Player Movement and Jumping. Movement using WASD is instant and very responsive without any slide effect after releasing the key. There is also a Directional Light at (50, -30, 0), which generates an exact shadow cast by the player’s capsule onto every surface, giving visual aids to determine platform distance. A press on the Space bar while the player is standing initiates jumping, which shows a parabolic trajectory in real time.

Output 3 — Coin Collection. There are six coins, which are made up of rotating gold spheres positioned at coordinates (0, 1, 5), (3, 2.75, 5), (0, 4, 5), (-3, 5.25, 5), (3, 5.25, 5), and (0, 6.5, 5) on Level 1. The sphere makes one rotation per second about the Y-axis.

Upon stepping on any of the coins, a chime sounds, and the coin disappears from sight, and the HUD coin counter is incremented by one.

Output 4 — Enemies' Patrols and Stomp. There are two enemies in Level 1; their color is red and their shape is a cube. These two enemies patrol on the floor level for about six units per patrol route at the speed of two units per second. If you jump on any one of the enemies, the following occurs: stomp sound is played, enemy disappears, score increases by ten points, and there is a minor bouncing up impulse applied to the player's capsule.

Output 5 — Completion of Level. Once all six coins are collected (or even enemies killed, if possible), the screen for Level Complete pops up: the message 'LEVEL COMPLETE!', followed by the line 'Final Score: [n]', indicating your score total, and finally a button labeled 'NEXT LEVEL', which proceeds to load Level 2.

8.3 Level 2 Gameplay Output

The Level 2 environment is visually different and more challenging in terms of the techniques used. The size of the floor is large, measuring 80 by 80 units. Also, platforms are thinner in width (measuring 1.5 units) and have increased levels from the previous level (up to 8 meters in height). Four enemy red cubes are found within the Level 2 environment, two on the ground floor with wider patrol routes of 10 units, and the other two located on higher platforms measuring 3.5 meters and 6.0 meters respectively.

Platform A and Platform B are the two novel game mechanics added to Level 2. Platform A travels between the coordinates (4, 2, 0) and (10, 2, 0) horizontally at a speed of 0.8 units of journeyFraction per second. Platform B, on the other hand, travels between (-4, 4, 0) and (-10, 4, 0) horizontally at a speed of 1.2 units per second. The players need to make their jump such that they hit each platform while they cross their location, move to the end point, and then jump to the next fixed platform.

The total number of coins in Level 2 is ten, out of which two are on the ground floor, three are on the static platforms located below, two are present on the top surfaces of the moving platforms, which requires the player to jump onto the platform and collect, and three coins are placed on the highest cluster of platforms that can be collected after making five precise jumps consecutively. The Level Complete screen at the end of Level 2 shows the total score of the first two levels.

8.4 Game Over Screen Output

The Game Over canvas is drawn on a full-screen semi-transparent overlay canvas with a black background showing a frozen scene of the current level where the player died. The words "GAME OVER" are shown in large red-colored text using the TextMeshPro asset, centrally aligned. "Score: [n]" shows the score when the player dies below the message. There are two options available: one for retrying ("RETRY") by calling the ResetGame() method, which resets the score to zero and then loads Level 1 again using the LoadScene() method; and another option for returning to the main menu by loading the StartScreen.

8.5 Build Output

This Windows standalone build was created using the File -> Build Settings -> Build PC, Mac & Linux Standalone option with Architecture set to x86_64. The resulting output includes: GameCraft_3DPlatformer.exe (game executable, size 1.2 MB), GameCraft_3DPlatformer_Data/ (Unity Data folder, size 68.4 MB – contains all scenes, scripting, audio, etc.) and MonoBleedingEdge/ (this is the .NET framework). Total folder size = 74.2 MB. Launching GameCraft_3DPlatformer.exe takes you directly to the Start Screen. Gameplay has been verified to be exactly the same as running in Editor Play mode in 3 full game runs.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

The GameCraft project has successfully met all ten objectives set out, resulting in a fully developed and well-designed 3D platform game in two levels using only free tools such as Unity Hub, Unity Editor 2022.3 LTS, and Visual Studio Community 2022, proving that a 3D game can be created within the parameters of an educational value-added course project without spending any money on commercial software and without requiring 3D modeling and game development experience.

The technical achievements of this project are many. The physics-based method for player character movement through velocity manipulation in FixedUpdate is effective, creating realistic player actions. The Physics.CheckSphere technique for grounding detection is effective, allowing one jump to be enforced. The LateUpdate method used to follow the camera allows for smooth camera following with no jittering. The Vector3.MoveTowards method for enemy patrolling results in consistent AI action at a controllable speed. The moving platform system with parenting of the player to the platform works to avoid the sliding issue that occurs with simpler kinematic platform systems.

GameManager singleton and DontDestroyOnLoad, along with the scene navigation approach through the use of SceneManager class, create an organized game state structure with a proper division between gameplay logic and scene-related matters. In terms of UI text, the pattern of UIManager singleton per scene combined with TextMeshPro creates a neat and readable interface. As far as script organization is concerned, twelve scripts are used and are consistent with the Unity principle of single responsibility.

The process of debugging covered in Chapter 6 offers an example of the most common mistakes that beginner programmers make while developing applications in Unity, which can be described as the following: component initialization mistakes (such as Is Trigger for collectibles and Continuous collision detection for objects moving quickly), execution order problems (LateUpdate used by camera scripts), inefficient coding (performance pattern error, caching GetComponent in Start), and wrong lifecycle control for a Singleton (using DontDestroyOnLoad together with instance guard).

The user acceptance test results show that the game can be easily played by students who have never designed games before, with all four reviewers able to finish Level 1 and three out of four able to complete Level 2. Since all four reviewers were very satisfied with the smoothness of controls and sound effects in the game, it can be assumed that the game feel has been made professional enough for a VAC project.

This project is a proof of the pedagogical merit of using game development as an instrument to teach the principles of software engineering. The real-world necessity of implementing frame rate independent simulation through Time.deltaTime, keeping game states consistent in different scenes using singleton pattern, decoupling game logic from sound handling via AudioManager abstraction, and managing multiple asynchronous tasks using coroutines makes the use of software engineering very tangible.

9.2 Future Work

The following areas that require improvement have been selected as potential development trends for the future implementations of the GameCraft project, taking into account the development process, testing results, and feedback from users:

Direction 1 — Models of 3D Characters and Their Animation with Skeletons. Primitive shaped characters such as the capsule shaped player and cubed shaped enemies will be substituted with 3D characters whose skeletons have been created using models downloaded from sources such as the Unity Asset Store, Mixamo, and Kenney.nl. The Unity Animator component and state machine of the animator controller will then switch between the animations of Idle, Run, Jump, Fall, and Death depending on the state and speed determined by the Player Movement and Enemy Patrol scripts.

Direction 2 — Enemy AI with the help of NavMesh. While the existing AI based on waypoint navigation will remain as it is for enemies who patrol around looking for the player, the navmesh system from Unity will also be employed so that the enemies can pursue and attack the player. Physics.Raycast() will be used to determine whether the enemy has a line-of-sight on the player. The destination of the NavMeshAgent will then change to player.position for chasing the player with NavMeshAgent.SetDestination(player.position).

Direction 3 — Checkpoint and Respawning System. The result of user acceptance testing showed that there was a high demand for mid-level checkpoints. This could be done by employing a trigger zone Checkpoint.cs file that would store the player's position in the game world into the GameManager when the player passes through the trigger zone. In addition, the player would respawn at the checkpoint position without losing their score and coins.

Direction 4 — Mobile Build with Touch Inputs. Setting up the build for Android through Unity's Android Build Support and integrating the touch inputs through Unity's input system package and on-screen Joystick through the use of a Joystick UI element could help launch the build on Android phones. In this regard, the script PlayerMovement.cs will have to adapt to use the output value from the Joystick control instead of the Input.GetAxis method and will feature an on-screen Jump button instead of the Space key.

Direction 5 — Effects and Post-Processing. The visual enhancements to be made with Unity's VFX Graph or the old Particle System would include effects such as those when coins are collected, explosions when enemies are defeated, and sparkles that follow the player as they move quickly. Unity's Post-Processing effects such as Bloom for the highlights on coins, Color Grading to set different atmospheres in levels, and Screen Space Ambient Occlusion for more realistic contact shadows can all be used in the level scene as Volumes.

Direction 6 — Unity's PlayerPrefs API allows you to create an easy-to-use key-value pair system that stores data across multiple game plays in the device where it is stored. Using PlayerPrefs to implement a high score system by writing PlayerPrefs.SetInt("HighScore", score) and reading the value from PlayerPrefs.GetInt("HighScore", 0) will make it possible for your start screen to show your highest ever score, adding a replay factor to the game.

REFERENCES

[1] Hocking, J. (2018). Unity in Action: Multiplatform Game Development in C# (2nd ed.). Manning Publications, New York.

[2] Thorn, A. (2014). Mastering Unity Scripting. Packt Publishing, Birmingham.

[3] Okita, A. (2015). Learning C# by Developing Games with Unity 5 (3rd ed.). Packt Publishing, Birmingham.

[4] Unity Technologies. (2023). Unity User Manual 2022.3 LTS. Unity Technologies. https://docs.unity3d.com/2022.3/Documentation/Manual/

[5] Unity Technologies. (2023). Unity Scripting API — Rigidbody. Unity Technologies. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/Rigidbody.html

[6] Unity Technologies. (2023). Unity Scripting API — Physics.CheckSphere. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/Physics.CheckSphere.html

[7] Unity Technologies. (2023). Unity Scripting API — SceneManager. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/SceneManagement.SceneManager.html

[8] Unity Technologies. (2023). Unity Scripting API — AudioSource. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/AudioSource.html

[9] Unity Technologies. (2022). Unity Scripting API — Vector3.MoveTowards. https://docs.unity3d.com/ScriptReference/Vector3.MoveTowards.html

[10] Microsoft Corporation. (2023). C# Programming Guide — .NET Documentation. https://learn.microsoft.com/en-us/dotnet/csharp/

[11] Unity Technologies. (2023). Physics Best Practices. Unity Documentation. https://docs.unity3d.com/Manual/PhysicsSection.html

[12] Unity Technologies. (2023). Audio Overview — Unity Manual. https://docs.unity3d.com/Manual/AudioOverview.html

[13] Unity Technologies. (2023). UI Toolkit — Unity Manual. https://docs.unity3d.com/Manual/UIBuilder.html

[14] Gregory, J. (2018). Game Engine Architecture (3rd Edition). CRC Press, Boca Raton.

[15] Nystrom, R. (2014). Game Programming Patterns. Genever Benning Publishers. https://gameprogrammingpatterns.com/

[16] Unity Technologies. (2023). TextMeshPro Documentation. Unity Technologies. https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.0/manual/

[17] Unity Technologies. (2023). Unity AR Foundation Manual. https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@5.0/manual/

[18] Unity Technologies. (2023). XR Interaction Toolkit Documentation. https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@2.3/manual/

[19] Panimalar Engineering College. (2026). Value Added Course Curriculum: Game Development with Unity. Department of Artificial Intelligence and Data Science, Chennai — 600 123.

[20] Smith, B. (2020). Unity in Action: Multiplatform Game Development in C#, 3rd ed. Manning Publications.